
Introducing Kokkos

Evan Drake Suggs
ASCEND Research Group - TN Tech
3/4/2026

What is Kokkos

- Kokkos is a programming model from Sandia National Laboratories
 - A programming model differs from a programming library in that it extends the language instead of simply adding functions
 - It manages data in parallel across different execution and memory spaces (i.e. CPU vs. GPU, main vs. GPU memory, etc)
 - It aims for performance portability and on-node communication
-

What does Kokkos run on

- Traditional nodes have their own memory along with a CPU, but some nodes might have accelerators like GPUs or might be special architectures
 - GPUs are now general purpose accelerator coprocessors
 - High-performance computers have many different compute nodes with complex memory structures, most of the time differing from one system to another
-

What Kokkos works with

- Kokkos provides an interface that can use backends for performance portability and on-node communication
 - Kokkos uses backend such as CUDA, OpenMP, HPX, and C++ threads
 - Kokkos's sibling, MDSPAN, has become part of the C++ standard library with C++23
 - MPI is a complement to Kokkos for cross-node communication
-

A few definitions

- CUDA: Nvidia's altered version of C/C++ for parallel programming on GPUs
 - HPX: A library targeting concurrency and parallelism standards of C++
 - OpenMP: Open Multi Processing, a library for programming in parallel
 - C++ threads: Native threading features in the C++ language
 - MDSpan: Multi-dimensional Span: A data management library in the C++23 standard. Resembles a pared down Kokkos and by the same developers
 - MPI: Message Passing Interface, a standard for communication between nodes
-

Why Kokkos?

- A programming model that works well with C++ and complex node architectures, including multiprocessors and accelerators
 - Instead of learning several low-level programming models, using Kokkos allows the programmer to focus on algorithms while Kokkos manages performance and productivity
 - Kokkos supports current and forthcoming systems that can vary quite a bit
 - We will treat OpenMP, Cuda, OpenCL, etc as low-level programming languages/notations
-

What is Performance Portability

Performance: Able to properly and fully utilize the resources of a system

Portability: Able to be moved between different systems with a minimum of new or changed code

Performance Portability: Able to properly and fully utilize resources while being able to change between systems

For example, coding for a system with Nvidia GPUs then having to switch to one without them

On-Node Communication

- Node = server = multiprocessor
 - An HPC cluster is composed of many nodes working together
 - While they have fast connections, a lot of work is still done on-node
 - Kokkos provides features to specify memory spaces (where data should be) and execution spaces (where code should work on the data), while still being general enough for portability
-

Initialize Kokkos Program

```
#include <Kokkos_Core.hpp>

int main(int argc, char** argv) {
    Kokkos::initialize( argc, argv );
    {
        // do something in scope
    }
    Kokkos::finalize();
    return 0;
}
```

The View Datatype

- The primary feature of Kokkos is its structure, the View, which can manage arrays, their datatype, layout in memory, and which type of memory/execution space it is in
 - The View is essentially a shared pointer wrapping an array, albeit a highly specialized one
 - Thinking of it another way, the View Datatype is to a traditional C array as a simple matrix is to a tensor
 - The idea of the View predates Kokkos, and was introduced in <https://arxiv.org/abs/1008.2909> and also exists in the C++ Ranges library
-

Kokkos View cont

- Copies, assignment and passing are shallow (that is they default to just be another name for an underlying array), unless the deep copy function is used
- The template asterisk is a dimension not a pointer or a regex!
- This means it is a 1 dimensional View of integers

```
View<int*> A("test", 8);  
View<int*> B = A;  
  
// means View<int[8]> B  
  
B(2) = 4;  
  
// A(2) == 4;
```

Basic Methods: Extent, data, label

- The size of individual dimensions is accessed with `.extent(i)`
 - The underlying array is accessible by the `.data()` method, but this is discouraged as it messes with reference counting
 - All managed Views have a name string for debugging, that can be accessed with `.label()`
 - Most other attributes must be accessed using a typedef
 - Deallocation is handled by reference counting
-

View Kokkos Program

```
#include <Kokkos_Core.hpp>
#include <iostream>

int main(int argc, char** argv) {
    Kokkos::initialize( argc, argv );
    {
        int n = 8;
        Kokkos::View<double*> check("check", n);
    }
    Kokkos::finalize();
    return 0;
}
```

Deep Copying

```
Kokkos::View<int**> A("A", n, m);  
Kokkos::View<int**> B("B", n, m);  
// Copy every element in View A to View B  
Kokkos::deep_copy(B, A);  
B(2, 2) = 5;  
// A(2, 2) != 5;
```

Copying Errors

```
Kokkos::View<int[5][10]> A("A");
```

```
Kokkos::View<int**> B("B", n, m);
```

```
// errors if A and B are not the same dimensions
```

```
Kokkos::deep_copy(B, A);
```

Subviews

Creates slices of the memory of a View, similar to those in Python math libraries, such as numPy

```
Kokkos::View<double***> View_t A("A", n, m, l);
```

```
auto B = Kokkos::subview(A,
```

```
    std::make_pair(2, 5), 6, Kokkos::ALL);
```

```
// similar to numpy, B = A[[2:5], 6, :]
```

```
// exclusive, dims are indices 2-4, row 6 and all of l dim
```

```
// or an range of 5-2 = 3, ending at index 4
```

Subviews

Make Pair creates a range for the subview's dimension, so `make_pair(2, 5)` is a range of $5-2 = 3$ and Kokkos interprets that as indices 2, 3 and 4.

`make_pair(0, 5)` will not be exclusive and will start at the 0th index to index 4.

```
View_t A("A", n, m, 1);
```

```
auto B = Kokkos::subview(A,
```

```
    make_pair(0, 5), make_pair(2, 8), Kokkos::ALL);
```

```
// similar to numpy, B = A[:, 2:8, :]
```

Unmanaged Views

```
// unmanaged View, less debug info and data management  
char* buf = {...};  
  
Kokkos::View<char*> A(buf, n);  
  
// A.data() == buf;
```

On-node computation

- The parallel features of Kokkos reproduce the basic features of most parallel programming libraries
 - In fact, it often uses libraries, such as OpenMP, as backends
 - These parallel features use functors or function objects, objects that implement the C++ () operator to act like functions
 - These are normally either structs with an overloaded () operator or lambdas, unnamed functions that act as objects
-

Parallel_for

- Kokkos::parallel_for()
- Takes a policy (a range or number of iterations) and a functor, along with an optional name parameter

```
Kokkos::View<int*> check("check",  
n);
```

```
Kokkos::parallel_for ("iterator", n,  
KOKKOS_LAMBDA(const int& i){  
    check(i) = i*i;  
});
```

Parallel_reduce

- Kokkos::parallel_reduce()
- Similar to for, but outputs a result based on the functor

```
struct Foo{
KOKKOS_INLINE_FUNCTION void
operator(const double& i, double&
lsum) const{
    lsum+=i;
}    };
Foo functor;
double result;
Kokkos::parallel_reduce (n,
                        functor, result);
```

Parallel_scan

- Kokkos::parallel_scan():
 - This handles what is known as a prefix scan, where elements are added in parallel then output to a View of the same size
- ```
//typedef ... View_t;
double result;
// pre = 1, 2, 3, 4 ...
View_t pre("pre", n);
View_t post("post", n);
Kokkos::parallel_scan(n,
 prefix_scan, result);
// post = 1, 3, 6, 10 ...
```
-

---

# Fence

- Kokkos::fence(): waits (blocks) until all functions finish
- This is important for functions that happen asynchronously, such as parallel tasks that might not all finish at the same time

```
Kokkos :: parallel_for(n, functor);
Kokkos::deep_copy(a, b);
```

```
Kokkos::fence();
```

---

---

# Basic Kokkos Program

```
#include <Kokkos_Core.hpp>
#include <iostream>

int main(int argc, char** argv) {
 Kokkos::initialize(argc, argv);
 {
 int n = 8;
 int result = 0;
 Kokkos::View<int*> check("check", n);
 // fill array
 Kokkos::parallel_reduce("parallel sum", check.extent(0), KOKKOS_LAMBDA(const int&
i, int& lsum){
 lsum+=check(i);
 }, result);
 std::cout << "Result: " << result << std::endl;
 }
 Kokkos::finalize();
 return 0;
}
```

---